CS202, Spring 2026

Homework 2 - AVL Trees and Heaps

Due: 23:55, 12/03/2026

Before you start your homework, please read the following instructions carefully:

FAILURE TO FULFIL ANY OF THE FOLLOWING REQUIREMENTS WILL RESULT IN A GRADE SCORE OF 0 (zero) WITHOUT ANY CHANCE OF REDEMPTION.

- See the course page for any late submission policies and Honor Code for Assignments.
- Upload your solutions in a single ZIP archive using the Moodle submission form. Name the file as **studentID_name_surname_hw2**.zip.
- Your ZIP archive should contain **only** the following files:
  - **Source Code:** Provide separate C++ source files for each question, named as **name_surname_q1.cpp** and **name_surname_q2.cpp**. Note: You may include auxiliary header (.h) or implementation (.cpp) files if necessary, but the main function for each question must reside in the files named above.
  - **Makefile:** Include a single Makefile that compiles your source code into the following two binaries: **scheduler** (for Question 1), **monitor** (for Question 2). You will lose points if your Makefile does not work.
  - **No Subfolders:** Your ZIP archive must be **flat**. Do not place your files inside folders or subdirectories within the ZIP. All .cpp, .h, and the Makefile must reside in the **root** of the archive.
  - Do not forget to put your name, student ID, and section number in all of these files. Comment on your implementation well. Add a header (see below) to the beginning of each file:

    ```
    /**
     * Title: AVL Trees and Heaps
     * Author: Name & Surname
     * ID: 12345678
     * Section: 1
     * Homework: 2
     * Description: Short description of your code
     */
    ```
  - Do not include any unnecessary files, such as auxiliary files generated by your preferred IDE.
  - Please do not use **Turkish** letters in your file and folder names.
- Your code must compile.
- Your code must be complete.

- You ARE NOT ALLOWED to use any data structure or algorithm-related function from the C++ standard template library (STL) or any other external libraries.
- Your code must run on the **dijkstra.cs.bilkent.edu.tr** server.
- For any question related to the homework, contact your TA: *klea.zambaku@bilkent.edu.tr*

Question 1: Aging task scheduler (60 Points)

You are managing a system with **M identical processors**. There are **N processes** that need to be executed. Each process has an arrival_time, a processing_time, and an initial_priority.

The Aging Rule: To prevent low-priority tasks from waiting forever, this system uses "aging". For every **1 second** a process spends in the wait queue, its **priority increases by 1**.

Effective Priority (EP): At any current time **T**, the priority of a process **i** is calculated as:

$$EP_i(T) = initial_priority_i + (T - arrival_time_i)$$

Develop an algorithm that gets a .txt file as an input and outputs a chronologically ordered list of tasks based on their processing times. For each task, the terminal should display the task ID, along with its specific start and completion timestamps. You should also display the program's total execution time.

The rules of execution:

- **Implementation:** You must implement your **heap** data structures manually. The use of external libraries is forbidden.
- **Selection:** When a processor becomes free, it must pick the task from the 'Wait Queue' with the **highest Effective Priority (EP)**. Assume the processes are already ordered by arrival time in your input file.
    - **Hint:** Keep one heap for the wait queue, and one heap for the processor timeline.
- **Tie-breaker:** If two tasks have the same EP, the one that **arrived first** must be picked.
- **Efficiency:** Your solution must run in **O(N log N)** time.
    - **Hint:** Do not iterate through the queue to update priorities every second. From the $EP_i(T)$ formula, $initial_priority_i$ - $arrival_time_i$ is always a constant. Since T is the same for all processes, the process with the largest difference (see above) will always be executed first.

- **Execution time:** Use the **<chrono>** library in C++ to measure performance and output the result in **milliseconds (ms)** at the end.

Command Line Interface

The program must take the input filename as a command-line argument. The file will contain the number of tasks (N) and the number of processors (M) as the first line. The following lines will contain the task ID, arrival time, duration, and its priority.

```
username@dijkstra:~>./scheduler <input_filename>
```

Example 1

Input.txt

```
4	1
0	10	10
1	5	50
9	5	55
15	5	100
```

Explanation

```
N=4 tasks, M=1 processor
Task 0: Arr=0, Dur=10, Prio=10
Task 1: Arr=1, Dur=5, Prio=50
Task 2: Arr=9, Dur=5, Prio=55
Task 3: Arr=15, Dur=5, Prio=100
```

Note: The numbers are separated by '\t' (tabs) while lines are separated by '\n' (new line). There is a '\n' at the last line.

Terminal output:

```
Process 0: Start=0, Finish=10
Process 1: Start=10, Finish=15
Process 3: Start=15, Finish=20
Process 2: Start=20, Finish=25
Total program execution time: 0.12 ms
```

Logic:
T=0: Process 0 arrives and starts immediately. (Finishes at T=10).
T=1: Process 1 arrives. It waits.
T=9: Process 2 arrives. It waits.

T=10: Process 0 finishes.

- Process 1 EP = 50 + (10 - 1) = 59.
- Process 2 EP = 55 + (10 - 9) = 56.
- Process 1 starts because it "aged" to a higher priority. (Finishes at T=15).

T=15: Process 3 arrives exactly as Process 1 finishes.

- Process 2 EP = 55 + (15 - 9) = 61.
- Process 3 EP = 100 + (15 - 15) = 100.
- Process 3 starts immediately because its initial priority is much higher than Process 2's aged priority.

T=20: Process 3 finishes. Process 2 starts and finishes at T=25.

Example 2

Input.txt

```
5     2
0     10    10
1     10    10
2     5     50
8     5     60
12    5     100
```

Explanation

```
N=5 tasks, M=2 processors
Task 0: Arr=0, Dur=10, Prio=10
Task 1: Arr=1, Dur=10, Prio=10
Task 2: Arr=2, Dur=5, Prio=50
Task 3: Arr=8, Dur=5, Prio=60
Task 4: Arr=12, Dur=5, Prio=100
```

Terminal output:

```
Process 0: Start=0, Finish=10
Process 1: Start=1, Finish=11
Process 3: Start=10, Finish=15
Process 2: Start=11, Finish=16
Process 4: Start=15, Finish=20
Total program execution time: 0.10 ms
```

Logic:

T=0: **Task 0** arrives. Processor A is free.

- Task 0 starts on Processor A (Finishes at T=10).

T=1: **Task 1** arrives. Processor B is free.

- Task 1 starts on Processor B (Finishes at T=11).

T=2: **Task 2** arrives. Both processors are busy.

- Task 2 enters the Wait Queue.

T=8: **Task 3** arrives. Both processors are busy.

- Task 3 enters the Wait Queue.

T=10: **Task 0 finishes** on Processor A. Processor A is now free.

- We check the Wait Queue at T=10:
  - Task 2 EP = 50 + (10 - 2) = 58.
  - Task 3 EP = 60 + (10 - 8) = 62.
- **Task 3 starts** on Processor A (Finishes at T=15).

T=11: **Task 1 finishes** on Processor B. Processor B is now free.

- Task 2 is in the queue.
- **Task 2 starts** on Processor B (Finishes at T=16).

T=12: **Task 4** arrives. Both processors are busy.

- Task 4 enters the Wait Queue.

T=15: **Task 3 finishes** on Processor A.

- **Task 4 starts** on Processor A (Finishes at T=20).

Question 2: Sequencing quality monitor (40 Points)

```
>gnl|SRA|SRR37295043.1.1   1   Biological (Biological)
CAGCCAATCT TTGGGGCAAT GGAACAAATT GCCGCCCATT GTGAGTCATA TTCGCCCTGA
CTTTCCAGAA CCATACGAAC TGCCCGTTGA CGGACCTCAG GGGAAAAACG TGTATTTTTA
GTCATCCTGT TTACCTCTTT CTCAGGGAGT T

One channel quality score
33 33 33 33 33 37 37 37 37 37  37 37 38 38 34 38 38 38 38 38  38 38 37 38 39 39 39 38 39 39
37 39 39 38 38 38 38 38 39 39  39 39 39 38 39 39 39 39 39 39  39 39 39 39 38 38 38 38 39 39
38 39 39 39 39 39 39 39 39 39  39 39 39 39 39 39 39 38 39 38  39 39 38 39 39 38 37 39 38 39
39 38 38 38 38 38 39 39 39 39  39 39 38 38 38 39 39 39 38 38  38 39 39 39 39 39 39 39 39 39
39 39 39 39 39 39 39 39 39 39  39 39 39 37 39 39 39 39 39 39  39 39 39 39 39 39 38 38 37 38
39
```

Fig 1: Actual example of Illumina reads for *E. coli* taken from NCBI. The problem below uses an adaptation of the real data.

In genomic sequencing, a **Quality Score** (an integer for this task) represents the reliability of each nucleotide base call. Sequences often contain "noisy" regions due to sequencing errors. To detect these, you will process a stream of (a total of) **N** quality scores using a **Sliding Window** of size **W**.
In each window, you will track the **Local Impact Sum** (the sum of the **P lowest** quality scores). Finally, you will report a **Global Quality Metric**, which is the **sum of all local impact sums** calculated throughout the entire sequence.

Because genomic data is massive, your analysis must be extremely efficient. You must achieve **O(N log W)** complexity. By storing **subtree_size** and **subtree_sum** in each node, you eliminate the need to traverse P nodes.

Requirements:

- You must implement a balanced AVL tree from scratch. The use of external libraries is forbidden.
- To calculate the sum of the **P lowest** scores in logarithmic time, every node must be **augmented** with:
  - **subtree_size**: Total number of nodes in that subtree.
  - **subtree_sum**: Mathematical sum of all values in that subtree.
  - **value_count**: Handles duplicates.
- As the window slides across the sequence:
  - **Calculate the Local Sum:** Find the sum of the **P smallest** elements in the current window.

- **Accumulate the Global Sum:** Maintain a running total of every Local Sum calculated.

Command Line Interface

The program must take the input filename as a command-line argument. The input file will contain the number of scores (N), the window size (W), and the P information as the first line. The following line will be the sequence of scores.

```
username@dijkstra:~>./monitor <input_filename>
```

Example:

Input.txt

```
10	4	3
15	5	20	10	25	2	8	30	1	12
```

Note: The numbers are separated by '\t' (tabs) while lines are separated by '\n' (new line). There is a '\n' at the last line.

Logic:

N=10 W=4 P=3

Step	Window Elements	Sorted Window	Sum of 3 Smallest	Local Sum
1	[15, 5, 20, 10]	5, 10, 15, 20	5 + 10 + 15	30
2	[5, 20, 10, 25]	5, 10, 20, 25	5 + 10 + 20	35
3	[20, 10, 25, 2]	2, 10, 20, 25	2 + 10 + 20	32

4	[10, 25, 2, 8]	2, 8, 10, 25	2 + 8 + 10	20
5	[25, 2, 8, 30]	2, 8, 25, 30	2 + 8 + 25	35
6	[2, 8, 30, 1]	1, 2, 8, 30	1 + 2 + 8	11
7	[8, 30, 1, 12]	1, 8, 12, 30	1 + 8 + 12	21

Terminal Output:

```
30 35 32 20 35 11 21
Global Sum: 184
```